Protocolo Experimental

Notificações e trabalho em segundo plano em Android nativo (Java e Kotlin)

Programação de Dispositivos Móveis 2026/27 · Tema 5 · Grupo 12 · Universidade da Beira Interior

1. Enquadramento teórico

Um dispositivo móvel tem bateria, memória e atenção do utilizador limitadas. Por isso o Android impõe regras estritas à execução de código quando a app não está visível (Doze, App Standby, limites de execução em segundo plano). Uma aplicação deve (i) **nunca bloquear a thread principal** – o ciclo de eventos da interface – e (ii) escolher o mecanismo certo para trabalho que tem de continuar, ou acontecer mais tarde, sem a interface.

Notificações. São o canal pelo qual uma app comunica com o utilizador fora da sua janela. Cada notificação pertence a um *canal* (API 26): é o utilizador, e não a app, quem controla a importância, o som e a vibração de cada canal. Desde o Android 13 (API 33) é necessária a permissão de execução `POST_NOTIFICATIONS`; sem ela as notificações ficam bloqueadas. Uma notificação ganha comportamento através de `PendingIntent`: um "testemunho" que dá ao sistema o direito de executar um Intent (abrir uma Activity, difundir um broadcast, iniciar um serviço) com a identidade da app. Desde o Android 12 é obrigatório declarar `FLAG_IMMUTABLE` ou `FLAG_MUTABLE`.

Trabalho em segundo plano. Não existe um único mecanismo. Trabalho que se perde sem problemas se o processo morrer pode usar threads/coroutines; trabalho *diferível e garantido* (mesmo após reiniciar a app ou o dispositivo) deve usar o **WorkManager**, que persiste os pedidos numa base de dados interna e delega a execução ao sistema (JobScheduler), respeitando restrições (rede, bateria, carregamento). Trabalho que o utilizador tem de ver a decorrer, durante muito tempo, usa um **Foreground Service** com notificação permanente e tipo declarado. Ações a uma hora precisa (despertador, calendário) usam o **AlarmManager**, cujos alarmes exatos exigem permissão especial e não sobrevivem a um reinício.

Estes mecanismos relacionam-se com a matéria das aulas: o ciclo de eventos (as operações demoradas não devem bloquear a interface), os componentes Activity, Service e BroadcastReceiver, o atributo `android:exported` e o Logcat. Os slides da aula de 21/09 referem que, para tarefas diferidas e persistentes, é normalmente preferível o WorkManager.

2. Objetivos

- Obter a permissão de notificações no contexto certo e tratar a sua recusa.
- Criar canais e publicar uma notificação com toque (*PendingIntent* de Activity) e ação interativa (*PendingIntent* de broadcast).
- Executar uma tarefa demorada fora da thread principal com WorkManager, observar o progresso e receber o resultado.
- Comparar três formas de agendar: trabalho único com restrições, trabalho periódico e alarme (AlarmManager).
- Observar o efeito de matar a app, de não cumprir restrições e de revogar a permissão.

3. Configuração utilizada

Item	Valor
IDE	Android Studio (versão estável atual); Gradle e AGP propostos pelo template
Projeto	Empty Views Activity · Linguagem Java · Package `pt.ubi.pdm.grupo12.background`
SDK	minSdk 29 · compileSdk/targetSdk propostos pelo template (≥ 33)
Biblioteca	`androidx.work:work-runtime:2.12.0` (estável em 01/11/2026)

Item	Valor
Dispositivos de teste	Emulador Pixel com API 34 ou superior (para ver a permissão) e, opcionalmente, telemóvel real
Ferramentas	Logcat; App Inspection > Background Task Inspector; adb
Segredos	Nenhum: a app não usa rede, contas nem chaves

4. Dados de entrada

Texto de exemplo (usado quando a caixa de texto está vazia), 161 caracteres:

```
Era uma vez uma história que ninguém sabia terminar. Cada escritor acrescentava uma frase, e a história crescia; a história mudava, e a história nunca terminava.
```

Resultado esperado da análise sobre este texto: **26 palavras, 18 distintas, 161 caracteres. Mais frequente: "história" (4x).** A tarefa simula trabalho demorado com 10 passos de 500 ms (≈ 5 s).

5. Procedimento

5.1 Preparação do projeto

1. Android Studio > New Project > Empty Views Activity. Name: G12 Segundo Plano; Package name: `pt.ubi.pdm.grupo12.background`; Language: Java; Minimum SDK: API 29.
2. Clonar o repositório do grupo. Apagar do projeto `MainActivity.java`, `res/layout/activity_main.xml` e `res/values/strings.xml` gerados.
3. Copiar o conteúdo de `app/src/main/` do repositório para o projeto (substitui também o `AndroidManifest.xml`).
4. No `app/build.gradle.kts` acrescentar `implementation("androidx.work:work-runtime:2.12.0")` (ver `app/DEPENDENCIES.txt`) e fazer Sync Project.
5. Executar (Run) no emulador. Abrir o Logcat e filtrar por `G12`.

5.2 Ensaios

#	Passos	Resultado esperado
T1 Permissão	Primeira execução em API 33+. Tocar em "1. Processar agora".	Aparece o diálogo de permissão de notificações. Permitir: toast "Notificações permitidas".
T2 Trabalho único	Caixa vazia. Tocar em "1. Processar agora" e observar.	Estado RUNNING com barra de progresso 10%...100% em ≈ 5 s. Notificação "Processamento concluído" com o resultado do ponto 4. Logcat: `G12Worker` "Início..." e "Fim..." (a thread não é `main`).
T3 Toque	Tocar no corpo da notificação.	A app abre (onNewIntent) e "Último resultado" mostra o texto da análise.
T4 Ação interativa	Repetir T2 e tocar no botão "Repetir" da notificação.	A notificação desaparece; Logcat `G12Receiver: onReceive: …ACTION_REPEAT`; novo trabalho corre e nova notificação surge.
T5 Restrições	Marcar "Só quando estiver a carregar". Em Extended Controls > Battery desligar o carregador (ou `adb shell dumpsys battery unplug`). Tocar em "2. Agendar com restrições".	Após 10 s o estado fica ENQUEUED (não executa). Ligar o carregador (`adb shell dumpsys battery set ac 1`): passa a RUNNING e conclui. No fim: `adb shell dumpsys battery reset`.
T6 Persistência	Tocar em "1. Processar agora" e, a meio, remover a app dos recentes.	O WorkManager volta a executar o trabalho (desde o início) e a notificação surge na mesma. Evidência: Logcat/Background Task Inspector.

#	Passos	Resultado esperado
T7 Periódico	Tocar em "3. Ativar tarefa periódica". Tocar outra vez.	Estado ENQUEUED com etiqueta "periodico"; segundo toque não duplica (política KEEP). Background Task Inspector mostra 1 trabalho periódico. (Opcional: forçar execução com `adb shell cmd jobscheduler run -f pt.ubi.pdm.grupo12.background <JOB_ID>`, obtendo o JOB_ID em `adb shell dumpsys jobscheduler`.)
T8 Alarme	Tocar em "4. Alarme...", enviar a app para segundo plano e bloquear o ecrã.	≈ 30 s depois (pode atrasar, é inexato) surge a notificação "Alarme disparado". Logcat: `ACTION_ALARM`.
T9 Permissão negada	`adb shell pm revoke pt.ubi.pdm.grupo12.background android.permission.POST_NOTIFICATIONS` e repetir T2.	O trabalho conclui, mas não há notificação. Logcat (aviso): "Notificações desativadas/negadas".
T10 Cancelamento	Com T7 ativo, tocar em "Cancelar todas as tarefas".	Lista de tarefas ativas fica vazia ("Sem tarefas ativas").
T11 (opcional) Doze	`adb shell dumpsys deviceidle force-idle`, repetir T8 e depois `adb shell dumpsys deviceidle unforce`.	O alarme pode ser diferido enquanto o dispositivo está em Doze.

6. Métricas

- Estado do WorkInfo (ENQUEUED, RUNNING, SUCCEEDED, CANCELLED) e percentagem de progresso.
- Tempo entre o toque e a notificação (esperado ≈ 5 s em T2; ≈ 15 s em T5 com carregador ligado).
- Nome da thread nos registos do Logcat (`G12Worker`) – deve ser uma thread do WorkManager, nunca `main`.
- Presença/ausência da notificação conforme a permissão (T1, T9).
- Número de trabalhos periódicos ativos (T7: exatamente 1).

7. Erros frequentes e resolução

Sintoma	Causa provável / solução
Não aparece qualquer notificação	Permissão negada ou canal silenciado nas definições da app; verificar em Definições > Apps > Notificações.
Erro "Unresolved reference R" ou classe não encontrada	O package do projeto não é exatamente `pt.ubi.pdm.grupo12.background`.
Erro de build em WorkManager	Falta a dependência (5.1.4) ou o Sync Project.
Crash ao tocar na notificação em API 31+	PendingIntent sem `FLAG_IMMUTABLE`/`FLAG_MUTABLE`.
Trabalho periódico não corre aos 15 min exatos	É esperado: o WorkManager não garante hora exata e o sistema pode diferir (Doze, bucket de standby).
Alarme não dispara à hora	É um alarme inexato por desenho; alarmes exatos exigem `SCHEDULE_EXACT_ALARM`/`USE_EXACT_ALARM`.

8. Resultados esperados (resumo)

Se os ensaios T1 a T10 produzirem os comportamentos indicados, fica demonstrado que: (a) o trabalho demorado não bloqueia a interface; (b) o WorkManager mantém o trabalho mesmo se a app for terminada; (c) as restrições adiam a execução até serem cumpridas; (d) as notificações dependem de permissão e de canais; (e) o AlarmManager serve para acordar a app a uma hora aproximada, mas não oferece persistência após reinício.

9. Referências

- Android Developers – Notifications overview: https://developer.android.com/develop/ui/views/notifications
- Android Developers – Notification runtime permission: https://developer.android.com/develop/ui/views/notifications/notification-permission
- Android Developers – WorkManager: https://developer.android.com/topic/libraries/architecture/workmanager
- Android Developers – Foreground services: https://developer.android.com/develop/background-work/services/fgs
- Android Developers – Schedule alarms: https://developer.android.com/develop/background-work/services/alarms
- Android Developers – Doze and App Standby: https://developer.android.com/training/monitoring-device-state/doze-standby
- AndroidX – WorkManager releases: https://developer.android.com/jetpack/androidx/releases/work
- P. Fazendeiro, slides de PDM 2026/27 (aulas de 16/09 e 21/09) e guiões dos Labs 4–7, UBI.